

TRAIVO

INTELLIGENT FIELD SERVICE MANAGEMENT

UPPDATERINGAR 2026-06-12

Viktiga ändringar och förtydliganden från Mats

Från: Tre konversationer med Mats — 12 juni 2026

For: Replit implementation

Ämnen: Metadata • Tidsfönster • Statusflöde • Filter • UI-ändringar

Prioritet: HÖG — implementera snarast

Status: Ny information — uppdaterar befintlig spec

Innehållsförteckning

Sida 1 — Översikt & Huvudsakliga ämnen	3
Sida 2–3 — Objekt & Metadata-system	4
Objekt-principen	4
Metadata-bibliotek	4
Arvsmekanik	5
Tidsfönster (positiva & negativa)	6
Sida 4 — Utökat Statusflöde (8 steg)	8
Sida 5 — Filter-uppdateringar	10
Nytt filter: Team	10
KRITISK ÄNDRING: Uppgiftstyp = Dropdown	11
Sida 6 — Ny UI-referensbild	13
Sida 7 — Implementation Checklista	14
1. Datamodell	14
2. API-endpoints	15
3. Frontend-komponenter	15
4. Testning	16
Sida 8 — Sammanfattning & Nästa steg	17

VIKTIGA UPPDATERINGAR — ÖVERSIKT

BAKGRUND

Denna PDF sammanfattar nya krav och förtydliganden från tre konversationer med Mats den 12 juni 2026. Innehållet uppdaterar och kompletterar den befintliga specifikationen (REPLIT_TRAIVO_KOMPLETT_SPEC.pdf). Läs noggrant — flera ändringar är kritiska för implementation.

Huvudsakliga ämnen

#	Ämne	Typ	Prioritet
1	Objekt & Metadata-system	Ny information	Hög
2	Tidsfönster (positiva/negativa)	Ny feature	Hög
3	Arvsmekanik för metadata	Ny information	Hög
4	Utökat statusflöde (8 steg)	Ändring	Hög
5	Nytt filter: "Team"	Ny feature	Normal
6	Uppgiftstyp = Dropdown (EJ checkboxar)	KRITISK ÄNDRING	KRITISK
7	Uppdaterad färgkodning för status	Ändring	Normal

Läsordning

1. Läs sammanfattningen — sida 2–5
2. Studera ny UI-bild — sida 6
3. Implementera ändringarna enligt checklista — sida 7
4. Verifiera implementationen mot checklistan

VIKTIG NOTERING

Dessa uppdateringar **ersätter och kompletterar** avsnitt i den ursprungliga specifikationen. Vid konflikt mellan detta dokument och den ursprungliga specen gäller **detta dokument**.

OBJEKT & METADATA-SYSTEM

Mats kärnprincip — Objekt-definitionen

FUNDAMENTAL PRINCIP

"Objekt har bara namn/nummer. Allt annat är metadata."

Detta är Mats exakta definition och ska styra hela datamodellen framöver.

Vad ett Objekt ÄR kontra VAD det HAR

Objekt (kärndata)	Metadata (allt annat)
Namn / Beteckning	Adress (longitud/latitud)
Intern ID	Tre-ords-adress
Externt ID (kund-ref)	Tidsfönster
—	Kontaktpersoner
—	Kommentarer / instruktioner
—	Kundspecifika attribut

Systemgenererade metadata-fält

Följande fält genereras automatiskt av systemet och läggs till som metadata:

- **Adress** — Longitud och latitud (via geocoding)
- **Tre-ords-adress** — What3Words eller liknande
- **Tidsfönster** — Kopplade önskade/spärrade tider

Metadata-bibliotek

Metadata kan definieras som ett "bibliotek" för att tillåta **flera värden på samma objekt**.

Praktiska exempel

Objekt	Metadata-typ	Flera värden
--------	--------------	--------------

Återvinningsrum BRF Solsidan	Tidsfönster	Måndag 08–10, Tisdag 14–16, Torsdag 08–12
ICA Maxi Sundsvall	Kontaktpersoner	Driftchef: Erik, Jourtelefon: 070-xxx, Nattjour: 073-xxx
Fastighet Fabriksgatan 1	Tillträdesinformation	Portkod, Nyckelskåpskod, Hissregler

IMPLEMENTATION — METADATA-BIBLIOTEK

Metadata-tabellen behöver stödja godtyckligt antal rader per objekt och per metadata-typ. Använd en EAV-modell (Entity-Attribute-Value) eller en JSONB-kolumn i PostgreSQL för flexibilitet.

Arvsmekanik för metadata

Metadata kan ärvas nedåt i objekthierarkin. Fält behöver bara anges på den högsta relevanta nivån och ärvs automatiskt till undernivåer.

Exempel: Adressarv

```
Återvinningsrum (adress: "Storgatan 22, Sundsvall")
├─ Maskin A (ärver adress från rum)
│   └─ Kär1 660L (ärver adress från maskin)
│       └─ Kär1 370L (ärver adress från maskin)

Fastighetsportal (adress: "Hamnvägen 4")
├─ Källarförråd 1 (ärver adress)
├─ Källarförråd 2 (ärver adress)
└─ Pannrum (EGNA koordinater – OVERRIDAR arv)
```

Arvsregler

Regel	Beskrivning
Standard: Ärv uppåt	Om fält saknas på undernivå — sök hos förälder, sedan far-farförälder etc.
Override: Eget värde	Om undernivå har eget värde används det (överridar arv)
Nullable arv	Null-värde på undernivå = "Ärv ej, lämna tomt" (skiljer sig från saknat fält)
canInherit-flagga	Vissa metadata-fält kan markeras som ej ärv-bara

Prisma-modell för MetadataField

```
model MetadataField {
  id                String    @id @default(cuid())
  tenantId          String
  objektId          String
  fieldKey          String    // t.ex. "adress", "kontaktperson"
  fieldValue        String
  inheritanceSource String?   // Varifrån värdet ärvs (objektId)
  canInherit        Boolean   @default(true) // Kan ärvas av barn
  isOverride        Boolean   @default(false) // Överridar förälders värde
  sortOrder         Int       @default(0)    // Ordning bland flera värden
  createdAt         DateTime  @default(now())

  objekt Objekt @relation(fields: [objektId], references: [id])
}
```

Tidsfönster — positiva och negativa

TYP 1: POSITIVA TIDSFÖNSTER (ÖNSKADE)

"Önskar leverans måndag 08:00–12:00"

- Anger ÖNSKAD tid
- Kan ha prioritet (Hög/Normal/Låg)
- Används vid ruttoptimering
- Varning om tid ej ryms

TYP 2: NEGATIVA TIDSFÖNSTER (SPÄRRADE)

"Får EJ levereras torsdag 14:00–16:00"

- Anger FÖRBJUDEN tid
- BLOCKERAR schemaläggning
- Systemet varnar om uppgift hamnar i spärrat fönster
- Gäller t.ex. störningskänsliga tider

Prisma-modell för Tidsfönster

```
model Tidsfönster {
  id          String   @id @default(cuid())
  tenantId    String
  objektId    String
  typ         String   // "positiv" | "negativ"
  veckodag    String?  // "måndag" | "tisdag" | ... | null = alla dagar
  startTid    String   // "08:00"
  slutTid     String   // "12:00"
  prioritet   String?  // "hög" | "normal" | "låg" (endast positiva)
  kommentar   String?  // Fritext, t.ex. "Ingen leverans under skolstarten"
  createdAt   DateTime @default(now())

  objekt Objekt @relation(fields: [objektId], references: [id])
}
```

Hantering i ruttoptimering

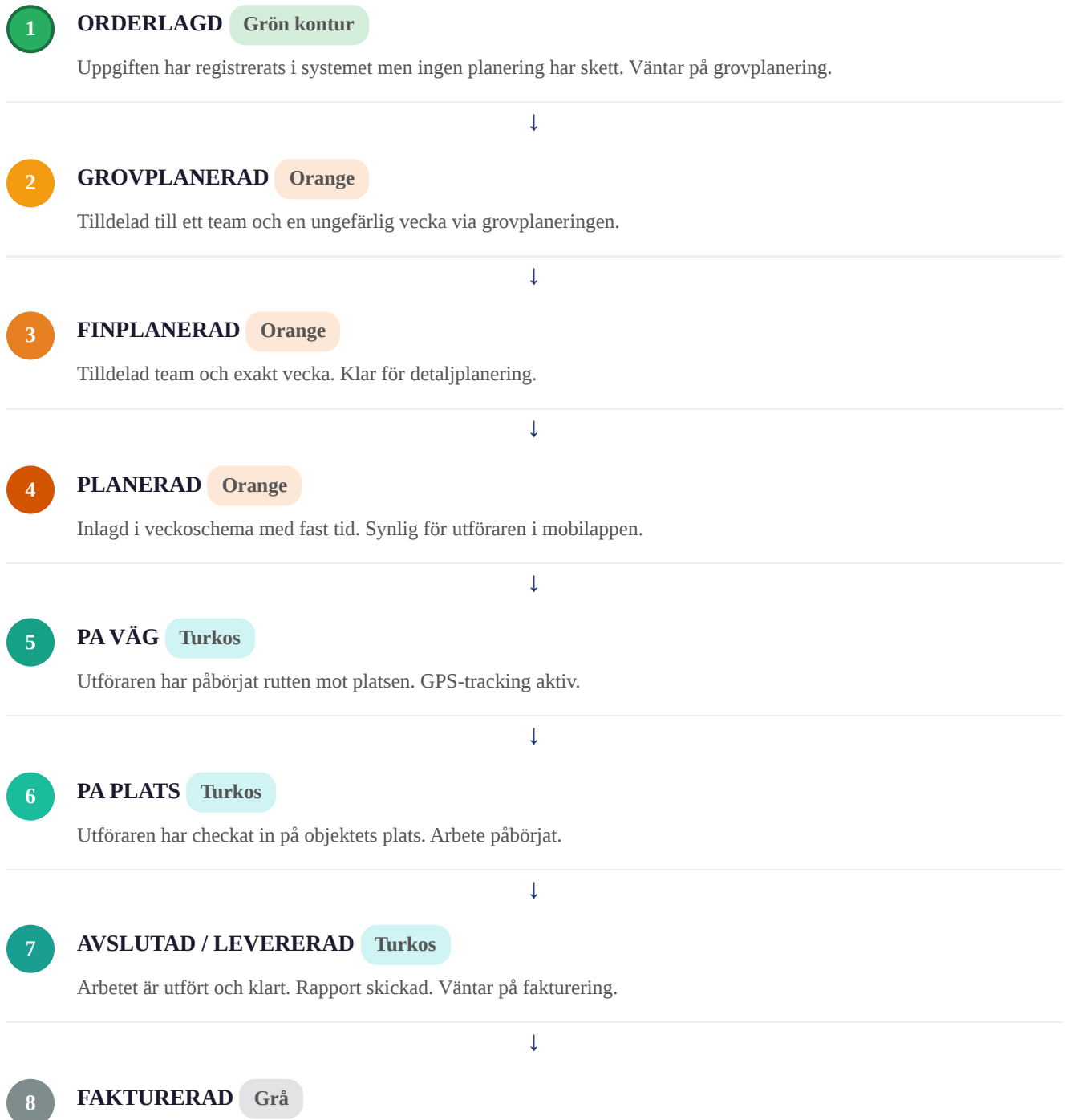
Fönstertyp	Algoritm-beteende	UI-indikation
Positivt (prioritet hög)	Placera uppgiften inom fönstret om möjligt, annars varning	Grön ram runt tidsblock
Positivt (prioritet normal)	Försök matcha, ingen varning om det misslyckas	Gul ram runt tidsblock
Negativt	Tillåt EJ placering i detta fönster	Röd skrafferingszon i kalendern

UTOKAT STATUSFLÖDE — 8 STEG

ÄNDRING FRÅN URSPRUNGLIG SPEC

Den ursprungliga specen hade 5 statusar. Mats har nu definierat ett mer granulärt 8-steps flöde inspirerat av moderna logistiksystem. **Uppdatera Uppgift-modellen och all färgkodning.**

Det fullständiga flödet



Ekonomiskt avslutad. Faktura skickad till kund. Arkiveras.

AVVIKARE — SEPARAT SPÅR

AVVIKER Röd är ett separat tillstånd (inte ett steg i flödet) som kan inträffa när som helst. Indikerar ett problem som kräver åtgärd. Uppgiften pausas tills avvikelsen är hanterad.

Fullständig färgkodning

Status	Steg	Färg	CSS-klass	Hex
Orderlagd	1	Grön kontur	border-green-500 bg-white	#22C55E (kontur)
Grovplanerad	2	Orange	bg-orange-500	#F97316
Finplanerad	3	Orange	bg-orange-600	#EA580C
Planerad	4	Orange	bg-orange-700	#C2410C
Pa väg	5	Turkos	bg-cyan-500	#06B6D4
Pa plats	6	Turkos	bg-cyan-600	#0891B2
Avslutad	7	Turkos	bg-teal-600	#0D9488
Fakturerad	8	Grå	bg-gray-400	#9CA3AF
Avviker	—	Röd	bg-red-500	#EF4444

TypeScript — statusColors-konstant

```
const statusColors: Record<string, string> = {
  orderlagd: "border-2 border-green-500 bg-white text-green-700",
  grovplanerad: "bg-orange-500 text-white",
  finplanerad: "bg-orange-600 text-white",
  planerad: "bg-orange-700 text-white",
  pa_vag: "bg-cyan-500 text-white",
  pa_plats: "bg-cyan-600 text-white",
  avslutad: "bg-teal-600 text-white",
  fakturerad: "bg-gray-400 text-white",
  avviker: "bg-red-500 text-white",
};
```

FILTER-UPPDATERINGAR

Nytt filter: Team

Filterpanelen ska utökas med ett nytt Team-filter som låter planeraren filtrera uppgifter per tilldelat team.

UI-layout

Team
[Lägg till team ▼]

– Dropdown öppnas vid klick –

- ☐ Team Norr 1
- ☐ Team Norr 2
- ☐ Team Syd
- ☐ DSU
- ☐ OJK-B0
- ☐ BHO
- ☐ ZML - B0
- ☐ YFA

Beteende

- Multi-select dropdown — kan välja flera team samtidigt
- Valda team visas som tags med ✕-knapp (som distrikt-filtret)
- Filtrerar på `uppgift.teamId IN [valda team]`
- Placeras i filterpanelen under Uppgiftsstatus-sektionen (se UI-bild på sida 6)

Ny API-endpoint

```
GET /api/teams?tenantId=xxx
```

```
Response: [
  { "id": "team-norr-1", "name": "Team Norr 1", "distrikt": "Norr" },
  { "id": "team-norr-2", "name": "Team Norr 2", "distrikt": "Norr" },
  { "id": "team-dsu",    "name": "DSU",          "distrikt": "Mitt" }
]
```

Uppdaterad filter-query

```
// Lägg till teamIds i filter-payload
POST /api/grovplanering/sok
Body: {
  distrikt: ["Distrikt Norr"],
  vecka: "2026-V24",
  uppgiftstyp: ["BÖK", "Service"],
  status: ["Otilldelad"],
  teamIds: ["team-norr-1", "team-norr-2"] // ← NYTT
}
```

KRITISK ÄNDRING: Uppgiftstyp = Dropdown

KRITISK ÄNDRING — ÅTGÄRDA OMEDELBART

UI-bilden (sida 6) visar fortfarande Uppgiftstyp som statiska checkboxar. **DETTA ÄR FEL.** Mats har tydliggjort att uppgiftstyper **MÅSTE** vara en dynamisk dropdown med multival.

FEL (SE BILDEN — SKA INTE GÖRAS)

Statiska checkboxar hårdkodade i koden:

- ☐ BÖK
- ☐ Service
- ☐ Tvätt
- ☐ Driftkontroll
- ☐ RBK
- ☐ Besiktning
- ☐ Administration
- ☐ Konsultation

RÄTT (MATS KRAV — GÖR SA HÄR)

Dynamisk dropdown hämtar från databas:

- Uppgiftstyp
[Välj uppgiftstyper ▼]
- Dropdown öppnas vid klick –
- ☐ BÖK
 - ☐ Service
 - ☐ Tvätt
 - ☐ ... (hämtas från API)

Varför dropdown?

Anledning	Förklaring
Dynamiska typer	Uppgiftstyper hämtas från ett sidoregister i databasen
Kundspecifika	Varje kund/tenant kan ha helt egna uppgiftstyper
Redigerbart under drift	Nya typer kan läggas till utan kod-ändringar
Internationalisering	Typer kan ha olika namn på olika språk

Ny API-endpoint: Dynamiska uppgiftstyper

GET /api/reference/task-types?tenantId=xxx

Response: [

```
{ "id": "1", "name": "BÖK",          "icon": "bin",      "active": true },
{ "id": "2", "name": "Service",      "icon": "wrench",  "active": true },
{ "id": "3", "name": "Tvätt",        "icon": "water",   "active": true },
{ "id": "4", "name": "Driftkontroll", "icon": "check",   "active": true },
{ "id": "5", "name": "RBK",          "icon": "recycle", "active": true },
{ "id": "6", "name": "Besiktning",    "icon": "search",  "active": true },
{ "id": "7", "name": "Administration", "icon": "doc",      "active": true },
{ "id": "8", "name": "Konsultation",  "icon": "people",  "active": true }
```

]

Komponent-implementation

```
// Hämta typer dynamiskt
const { data: taskTypes } = useQuery(
  ['task-types', tenantId],
  () => fetch(`/api/reference/task-types?tenantId=${tenantId}`).then(r => r.json())
);

// Dropdown-komponent
<MultiSelectDropdown
  label="Uppgiftstyp"
  placeholder="Välj uppgiftstyper"
  options={taskTypes?.map(t => ({ value: t.id, label: t.name })) ?? []}
  value={filter.uppgiftstyp}
  onChange={(values) => setFilter(f => ({ ...f, uppgiftstyp: values }))}
/>
```

NY UI-REFERENSBILD — UPPDATERAD

FILTERVY

LÄS DENNA BILDTEXT INNAN DU STUDERAR BILDEN!

Bilden nedan är en NY uppdaterad filtervvy som visar nya features. OBS: Bilden har ett fel (Uppgiftstyp visas som checkboxar). Läs anteckningarna noggrant.

The screenshot shows a filter view interface with the following elements:

- Filters:**
 - Distrikt:** Dropdown menu with "Lägg till distrikt".
 - Postnummer:** Input field with "t.ex. 802".
 - Ort:** Dropdown menu with "Alla orter".
 - Tidsperiod (önskad leveranstid):** Buttons for "Månad", "Vecka" (selected), and "Datumintervall", with a range "2026 - V24".
 - Uppgiftstyp:**
 - BÖK, Service, Tvätt, Administration
 - RBK, Driftkontroll, Besiktning, Konsultation
 - Uppgiftsstatus:**
 - Utförd (blue dot), Otilldelad (grey dot), Avviker (red dot)
 - Tilldelad (orange dot), Delvis utförd (green dot)
 - Team:** Dropdown menu with "Lägg till team".
- Buttons:** "Fler filter", "Rensa filter", "Filtrera".
- Grouping:** "Gruppera per: Objekt (selected), Kund, Orderkoncept, Ingen". Buttons: "0 markerade", "Markera alla", "Avmarkera alla", "Markera grupp", "Återkalla tilldelning", "Tilldela markerade".
- Table:**

	Status	Kund	Objekt / Uppgift	Uppgiftstyp	Önskad lev.	Tid	Team	Vecka	Senast utförd	Ordervärde
☐	▼	Inget objekt	0 objekt	1 uppgifter	2026-06-05	1 h				610 kr
☐	● Tilldelad	Telgebostäder AB	Kärntvätt standard	Service	2026-06-05	1 h	DSU	V24	-	610 kr
- Footer:** "Visar 1-1 av 1 uppgifter", "20/sida", and a "+" button.

UI-REFERENS: UPPDATERAD FILTERVY — 2026-06-12

NYTT I DENNA BILD (implementera):

- Team-filter (dropdown längst ner i filterpanelen)
- Färgkodade status-ikoner: turkos (Utförd), orange (Tilldelad), röd (Avviker), grön (Otilldelad), lila (Delvis utförd)
- Gruppera per: Objekt / Kund / Orderkoncept / Ingen
- Batch-åtgärder: Markera alla, Avmarkera, Markera grupp, Tilldela, Återkalla
- Kolumner: Status, Kund, Objekt/Uppgift, Uppgiftstyp, Önskad lev., Tid, Team, Vecka, Senast utförd, Ordervärde

FEL I BILDEN (korrigera):

- Uppgiftstyp visas som statiska checkboxar
- Ska ISTÄLLET vara en dropdown med multival
- Se sida 5 i detta dokument för korrekt implementation

FEL: Checkboxar for Uppgiftstyp

RÄTT: Dropdown med multival

Följ bilden för: Layout, kolumnordning, statusfärger, gruppering, batch-knappar — ALLT stämmer utom Uppgiftstyp-filtret.

IMPLEMENTATION CHECKLISTA

INSTRUKTION

Nedan listas alla ändringar som ska implementeras. Arbeta igenom listan uppifrån och ner. Bocka av varje punkt innan du går vidare till nästa.

1. Datamodell

Ny modell: Tidsfönster

- ☐ Skapa `Tidsfönster` -modell i Prisma schema (se sida 2–3)
- ☐ Fält: `id`, `tenantId`, `objektId`, `typ`, `veckodag`, `startTid`, `slutTid`, `prioritet`, `kommentar`
- ☐ Relation: `Objekt` (many `Tidsfönster` per `Objekt`)
- ☐ Migration: `prisma migrate dev --name add-tidsfönster`

Ny modell: MetadataField

- ☐ Skapa `MetadataField` -modell i Prisma schema (se sida 2–3)
- ☐ Fält: `id`, `tenantId`, `objektId`, `fieldKey`, `fieldValue`, `inheritanceSource`, `canInherit`, `isOverride`, `sortOrder`
- ☐ Migration: `prisma migrate dev --name add-metadata-field`

Uppdatera Uppgift-modell: Nytt statusflöde

```
// Uppdatera status-fältet i Uppgift
model Uppgift {
  ...
  status String @default("orderlagd")
  // Giltiga värden: "orderlagd" | "grovplanerad" | "finplanerad" |
  //                  "planerad" | "pa_vag" | "pa_plats" |
  //                  "avslutad" | "fakturerad" | "avviker"
}
```

- ☐ Uppdatera status-fältet i Uppgift-modellen
- ☐ Uppdatera seed-data med nya statusvärden
- ☐ Uppdatera alla API-svar som returnerar status
- ☐ Uppdatera migration

Ny modell: TaskType (sidoregister för uppgiftstyper)

```
model TaskType {
  id          String   @id @default(cuid())
  tenantId    String
  name        String   // "BÖK", "Service" etc.
  icon        String?  // Ikonnamn
  active      Boolean  @default(true)
  sortOrder   Int      @default(0)
  createdAt   DateTime @default(now())

  tenant      Tenant   @relation(...)
  uppgifter   Uppgift[]
}
```

- Skapa `TaskType` -modell i Prisma schema
- Uppdatera `Uppgift` att referera till `taskTypeId` istället för fritext
- Seed med standard-typer: BÖK, Service, Tvätt, Driftkontroll, RBK, Besiktning, Administration, Konsultation

2. API-endpoints

- Ny endpoint: `GET /api/teams?tenantId=xxx`
- Ny endpoint: `GET /api/reference/task-types?tenantId=xxx`
- Ny endpoint: `GET /api/objects/:id/time-windows`
- Ny endpoint: `POST /api/objects/:id/time-windows` (skapa/uppdatera tidsfönster)
- Uppdatera: `POST /api/grovplanering/sok` — lägg till `teamIds` som query-param
- Uppdatera: Alla endpoints som returnerar status — använd nya statuskoder
- Ny endpoint: `GET /api/metadata/:objektId` — hämta metadata med arvsmechanik

3. Frontend-komponenter

- **Nytt:** `TeamFilter` -komponent (multi-select dropdown)
- **KRITISK ÄNDRING:** Ersätt `TaskTypeCheckboxes` med `TaskTypeDropdown` (hämtar från API)
- **Uppdatera:** `StatusBadge` — implementera alla 9 statusvärden med rätt färgkoder
- **Nytt:** `TidsfönsterEditor` -komponent (lägg till/ta bort positiva och negativa tidsfönster)
- **Uppdatera:** `FilterPanel` — lägg till Team-filter och ersätt checkboxar

- **Uppdatera:** `FilterState` interface — lägg till `teamIds: string[]`

StatusBadge-komponent

```
function StatusBadge({ status }: { status: string }) {
  const config: Record<string, { label: string; className: string }> = {
    orderlagd: { label: "Orderlagd", className: "border-2 border-green-500 bg-white text-green-700" },
    grovplanerad: { label: "Grovplanerad", className: "bg-orange-500 text-white" },
    finplanerad: { label: "Finplanerad", className: "bg-orange-600 text-white" },
    planerad: { label: "Planerad", className: "bg-orange-700 text-white" },
    pa_vag: { label: "Pa väg", className: "bg-cyan-500 text-white" },
    pa_plats: { label: "Pa plats", className: "bg-cyan-600 text-white" },
    avslutad: { label: "Avslutad", className: "bg-teal-600 text-white" },
    fakturerad: { label: "Fakturerad", className: "bg-gray-400 text-white" },
    avviker: { label: "Avviker", className: "bg-red-500 text-white" },
  };
  const { label, className } = config[status] ?? config.orderlagd;
  return (
    <span className={`px-2 py-0.5 rounded-full text-xs font-bold ${className}`}>
      {label}
    </span>
  );
}
```

4. Testning

- Testa Team-filter: välj ett team → bara tilldelade uppgifter visas
- Testa dynamiska uppgiftstyper: lägg till ny typ i databasen → visas i dropdown
- Testa alla 9 statusfärger i tabellen
- Testa positiva tidsfönster: uppgift matchas korrekt med rätt fönster
- Testa negativa tidsfönster: system varnar om uppgift hamnar i spärrat fönster
- Testa arvsmekanik: undernivå utan adress ärver korrekt från förälder
- Testa override: undernivå med eget värde overridear förälderns
- Testa multi-select: välj flera team + flera uppgiftstyper simultant

SAMMANFATTNING & NÄSTA STEG

Viktiga beslut från Mats — snabbguide

1

Objekt = Namn/Nummer

Allt annat är metadata. Adress, tidsfönster, kontaktuppgifter — allt lagras separat som metadata med arvsmekanik.

2

Tidsfönster

Positiva (önskade tider) och negativa (spärrade tider). Styr ruttoptimering och ger varningar vid krockar.

3

8-steps statusflöde

Orderlagd → Grovplanerad → Finplanerad → Planerad → På väg → På plats → Avslutad → Fakturerad. Plus avviker som separat tillstånd.

4

KRITISK: Dropdown för Uppgiftstyp

MÅSTE vara dynamisk dropdown från sidoregister. Absolut INTE statiska checkboxar. Typer kan variera per kund och ändras under drift.

5

Nytt Team-filter

Multi-select dropdown. Filtrerar uppgifter per tilldelat team. Placeras i filterpanelen under Uppgiftsstatus.

6

Arvsmekanik

Metadata ärvs nedåt i objekthierarkin. Undernivåer kan överrida med eget värde. canInherit-flagga styr beteendet.

Nästa steg — exakt ordning

- 1] Läs denna PDF en gång till — säkerställ att du förstår alla 6 beslut ovan
- 2] Uppdatera Prisma-schemat: lägg till Tidsfönster, MetadataField och TaskType-modellerna
- 3] Migrationer och uppdatera seed-data med nya statusvärden och TaskTypes
- 4] Implementera nya API-endpoints (teams, task-types, time-windows)
- 5] Uppdatera befintliga endpoints med teamIds-filter och nya statuskoder
- 6] Byt Uppgiftstyp-checkboxar till dynamisk dropdown i FilterPanel
- 7] Lägg till Team-filter-komponenten i FilterPanel

8] Implementera StatusBadge med alla 9 statusvärden och färgkoder

9] Gör allt enligt checklistan på sida 7

10] Fråga om något är oklart INNAN du implementerar!

VIKTIGASTE PUNKTEN ATT MINNAS

Uppgiftstyp-filtret MÅSTE vara en dropdown. Detta är ett arkitekturellt beslut från Mats — uppgiftstyper är dynamiska och kund-specifika. Statiska checkboxar är fel oavsett hur enklare de är att implementera.

FRÅGOR?

Om något i denna spec är oklart — fråga INNAN du implementerar. Det är alltid bättre att fråga en gång för mycket än att implementera fel och behöva göra om.

Dokument: REPLIT_UPPDATERINGAR_2026-06-12.pdf • Version: 1.0 • Datum: 2026-06-12 • Baserat på: session_2026-06-12T06-48, session_2026-06-12T07-03, Fortsättning.json